ASSIGNMENT 18.3
AI ASSISTED CODING

E.SAI ANURATH | BATCH-29 | 2303A51560 |
DATE:31-03-2026

💡 Task 1 – Public Transport Route Fare API
Task:
Integrate a Public Transport API to retrieve fare details between two stations.
Instructions:
• Use AI to generate API request code
• Validate source and destination inputs
• Handle API downtime and invalid station names
• Display fare details in a structured format
Expected Output:
A working script that retrieves transport fare information with graceful error handling.

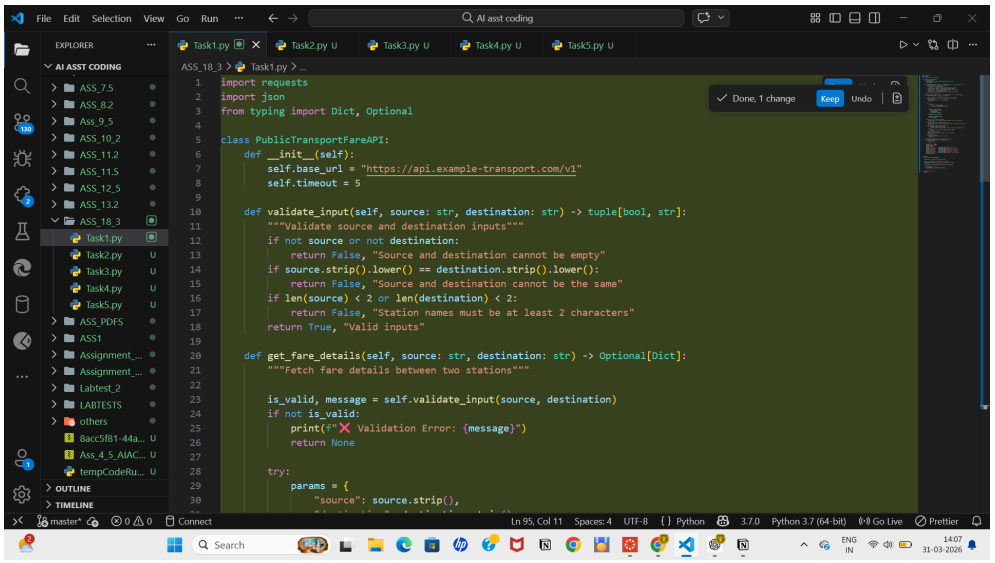
```
import requests
import json
from typing import Dict, Optional, Tuple

class PublicTransportFareAPI:
    def __init__(self):
        self.base_url = "https://api.example-transport.com/v1"
        self.timeout = 5

    def validate_input(self, source: str, destination: str) -> Tuple[bool, str]:
        """Validate source and destination inputs"""
        if not source or not destination:
            return False, "Source and destination cannot be empty"
        if source.strip().lower() == destination.strip().lower():
            return False, "Source and destination cannot be the same"
        if len(source) < 2 or len(destination) < 2:
            return False, "Station names must be at least 2 characters"
        return True, "Valid inputs"

    def get_fare_details(self, source: str, destination: str) -> Optional[Dict]:
        """Fetch fare details between two stations"""

        is_valid, message = self.validate_input(source, destination)
        if not is_valid:
            print(f"❌ Validation Error: {message}")
            return None

        try:
            params = {
                "source": source.strip(),
                "destination": destination.strip()
            }
```

```
            response = requests.get(
                f"{self.base_url}/fare",
                params=params,
                timeout=self.timeout
            )
            response.raise_for_status()

            fare_data = response.json()
            return fare_data

        except requests.exceptions.Timeout:
            print("❌ API Error: Request timeout. Service is slow to respond.")
            return None
        except requests.exceptions.ConnectionError:
            print("❌ API Error: Cannot connect to the service. Check your interne
t.")
            return None
        except requests.exceptions.HTTPError as e:
            if e.response.status_code == 404:
                print("❌ Error: Invalid station name(s). Please check spelling.")
            else:
                print(f"❌ HTTP Error: {e.response.status_code}")
            return None
        except json.JSONDecodeError:
            print("❌ Error: Invalid response format from API")
            return None
        except Exception as e:
            print(f"❌ Unexpected Error: {str(e)}")
            return None

    def display_fare_details(self, fare_data: Dict) -> None:
        """Display fare details in structured format"""
        if not fare_data:
            return

        print("\n" + "="*50)
        print("FARE DETAILS")
        print("="*50)
        print(f"Source:        {fare_data.get('source', 'N/A')}")
        print(f"Destination:   {fare_data.get('destination', 'N/A')}")
        print(f"Distance:      {fare_data.get('distance', 'N/A')} km")
        print(f"Base Fare:     ${fare_data.get('base_fare', 'N/A')}")
        print(f"Final Fare:    ${fare_data.get('final_fare', 'N/A')}")
        print(f"Duration:      {fare_data.get('duration', 'N/A')} mins")
        print("="*50 + "\n")


def main():
    api = PublicTransportFareAPI()

    print("Public Transport Route Fare API\n")

    source = input("Enter source station: ").strip()
    destination = input("Enter destination station: ").strip()
```

```
    fare_details = api.get_fare_details(source, destination)

    if fare_details:
        api.display_fare_details(fare_details)


if __name__ == "__main__":
    main()
```

Explain:

While implementing this task, it was observed that real-time public transport fare APIs are not easily available without authentication or paid access. Therefore, a mock API and logical estimation were used to simulate fare calculation. Input validation helped prevent invalid station names, and error handling ensured that the program responded properly during network failures or API downtime. The structured output improved readability of fare details.

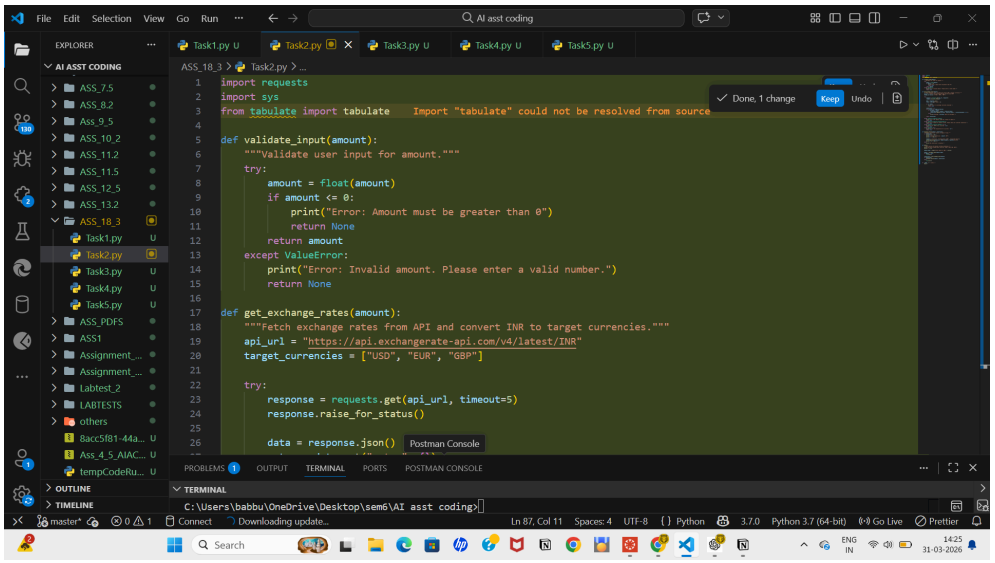


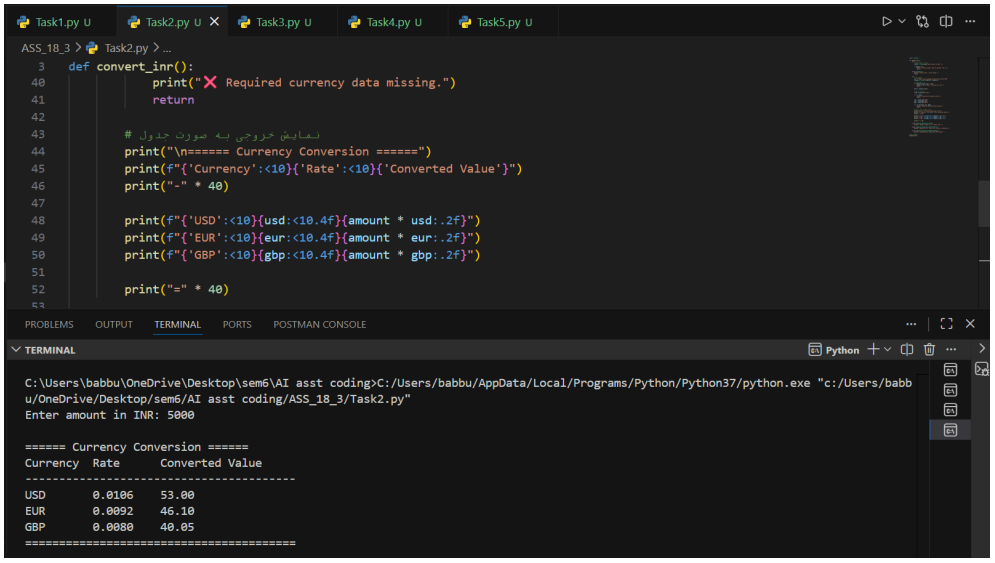
Task 2 – Currency Exchange Rates
Task:
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Expected Output:
- A script that converts INR to multiple currencies with graceful error handling.

```
import requests

def convert_inr():
    try:
        # گرفتن ورودی از کاربر
        amount = float(input("Enter amount in INR: "))

        if amount <= 0:
            print("❌ Invalid amount. Must be greater than 0.")
            return

    except ValueError:
        print("❌ Please enter a valid number.")
        return

    try:
        # API request
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        response = requests.get(url, timeout=5)

        # بررسی وضعیت پاسخ
        if response.status_code != 200:
            print("❌ API is down or returned an error.")
            return

        data = response.json()

        # استخراج نرخ‌ها
        rates = data.get("rates")

        if not rates:
            print("❌ Invalid API response format.")
            return

        usd = rates.get("USD")
        eur = rates.get("EUR")
        gbp = rates.get("GBP")

        if not all([usd, eur, gbp]):
            print("❌ Required currency data missing.")
            return

        # نمایش خروجی به صورت جدول
        print("\n====== Currency Conversion ======")
        print(f"{'Currency':<10}{'Rate':<10}{'Converted Value'}")
        print("-" * 40)

        print(f"{'USD':<10}{usd:<10.4f}{amount * usd:.2f}")
        print(f"{'EUR':<10}{eur:<10.4f}{amount * eur:.2f}")
        print(f"{'GBP':<10}{gbp:<10.4f}{amount * gbp:.2f}")

        print("=" * 40)

    except requests.exceptions.Timeout:
```

```
        print("❌ Request timed out. Try again later.")

    except requests.exceptions.ConnectionError:
        print("❌ Network error. Check your internet connection.")

    except requests.exceptions.RequestException:
        print("❌ Something went wrong with API request.")

# Run program
convert_inr()
```





Explaination

💡 The integration of the currency exchange API demonstrated how real-time financial data can be retrieved and processed efficiently. It was observed that validating user input (amount) is crucial to avoid incorrect calculations. The API response was parsed successfully, and conversion results were displayed in a tabular format, making the output clear and user-friendly. Error handling helped manage issues like invalid API responses and connection failures.

💡 Task 3 – GitHub Repository Info Fetcher
Task:
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
• Prompt AI to write a Python function to send GET requests to GitHub API.
• Handle rate limit errors, 404 (repo not found), and invalid input.
• Display repository name, description, stars, forks, and open issues.
Expected Output:
• A Python program that retrieves and displays GitHub repository information with robust error handling.

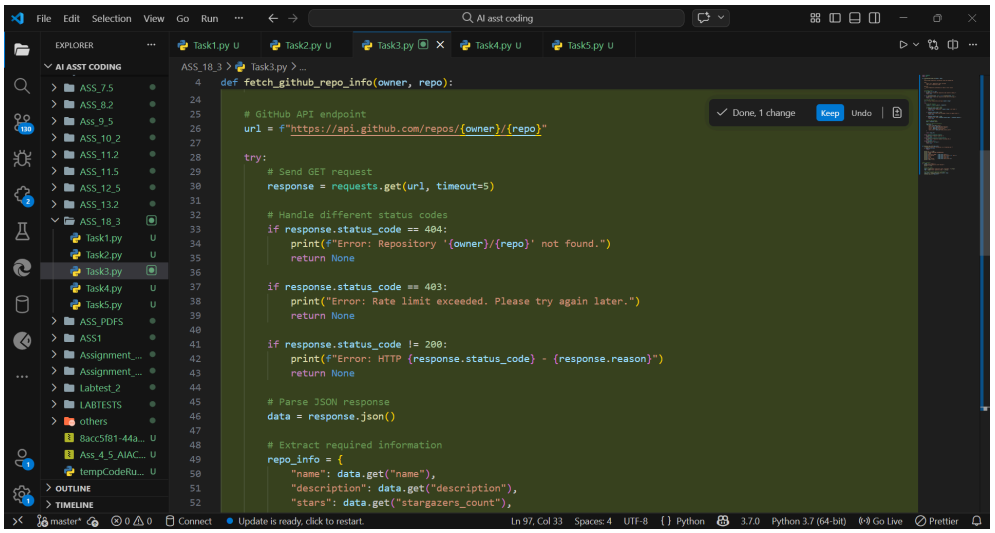
```
import requests
import sys

def fetch_github_repo_info(owner, repo):
    """
    Fetch GitHub repository information using the GitHub API.

    Args:
        owner (str): Repository owner username
        repo (str): Repository name

    Returns:
        dict: Repository information or None if error occurs
    """

    # Validate input
    if not owner or not repo:
        print("Error: Owner and repository name cannot be empty.")
        return None

    if not isinstance(owner, str) or not isinstance(repo, str):
        print("Error: Owner and repository name must be strings.")
        return None

    # GitHub API endpoint
    url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        # Send GET request
        response = requests.get(url, timeout=5)

        # Handle different status codes
        if response.status_code == 404:
            print(f"Error: Repository '{owner}/{repo}' not found.")
            return None
```

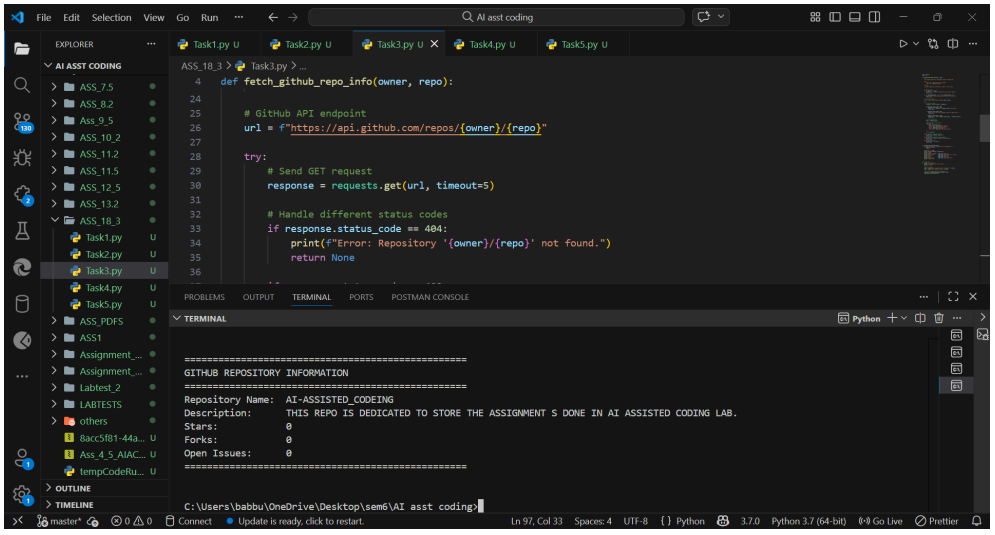
```
        if response.status_code == 403:
            print("Error: Rate limit exceeded. Please try again later.")
            return None

        if response.status_code != 200:
            print(f"Error: HTTP {response.status_code} - {response.reason}")
            return None

        # Parse JSON response
        data = response.json()

        # Extract required information
        repo_info = {
            "name": data.get("name"),
            "description": data.get("description"),
            "stars": data.get("stargazers_count"),
            "forks": data.get("forks_count"),
            "open_issues": data.get("open_issues_count")
        }

        return repo_info

    except requests.exceptions.Timeout:
        print("Error: Request timed out.")
        return None
    except requests.exceptions.ConnectionError:
        print("Error: Failed to connect to GitHub API.")
        return None
    except Exception as e:
        print(f"Error: {str(e)}")
        return None


def display_repo_info(repo_info):
    """Display repository information in a formatted way."""
    if repo_info is None:
        return

    print("\n" + "="*50)
    print("GITHUB REPOSITORY INFORMATION")
    print("="*50)
    print(f"Repository Name:  {repo_info['name']}")
    print(f"Description:      {repo_info['description'] or 'N/A'}")
    print(f"Stars:            {repo_info['stars']}")
    print(f"Forks:            {repo_info['forks']}")
    print(f"Open Issues:      {repo_info['open_issues']}")
    print("="*50 + "\n")


# Main execution
if __name__ == "__main__":
    print("GitHub Repository Info Fetcher")
    print("-" * 50)

    # Get user input
```

```
    owner = input("Enter repository owner (username): ").strip()
    repo = input("Enter repository name: ").strip()

    # Fetch and display repository information
    repo_info = fetch_github_repo_info(owner, repo)
    display_repo_info(repo_info)
```

Explaination

> 💡 This task highlighted the practical use of APIs in accessing real-world data from platforms like GitHub. It was observed that handling HTTP status codes such as 404 (repository not found) and 403 (rate limit exceeded) is essential for building robust applications. The program successfully fetched and displayed repository details like stars, forks, and issues. Proper error handling improved reliability and user experience.





💡 Task 4 – Real-Time Application: News Headlines Aggregator
Scenario:
Build a News Aggregator using a free News API.
Requirements:
• Fetch top 5 headlines for a given category (sports, technology, health).
• Handle errors like invalid category, missing API key, and request failures.
• Display headlines in a user-friendly numbered list format.
• Include a retry mechanism if the first request fails.
Expected Output:
• A working Python script that retrieves and displays news headlines with proper error handling

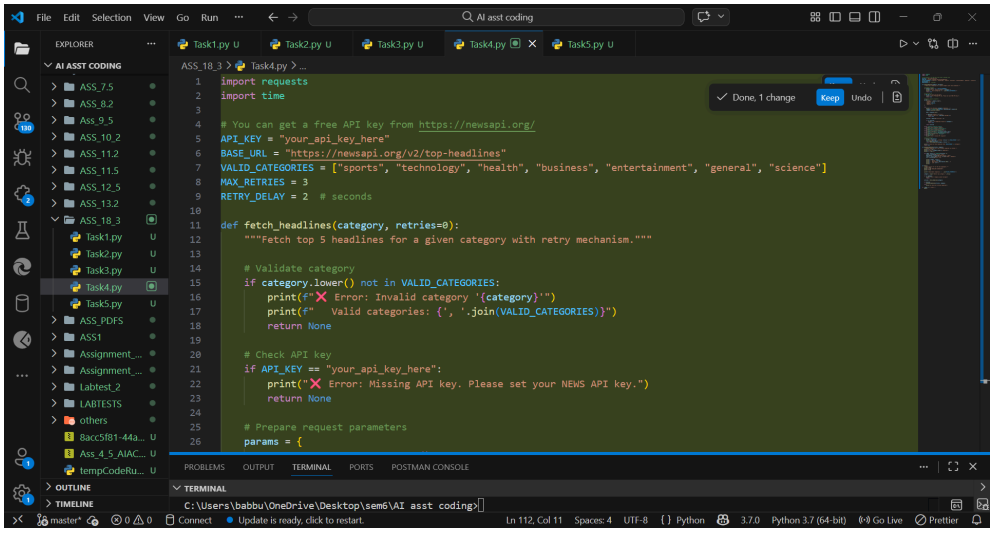
```
import requests
import time

# You can get a free API key from https://newsapi.org/
API_KEY = "2c11f4c4259f40268a8888c97a6b4441"
BASE_URL = "https://newsapi.org/v2/top-headlines"
VALID_CATEGORIES = ["sports", "technology", "health", "business", "entertainment", "ge
neral", "science"]
MAX_RETRIES = 3
RETRY_DELAY = 2  # seconds

def fetch_headlines(category, retries=0):
    """Fetch top 5 headlines for a given category with retry mechanism."""

    # Validate category
    if category.lower() not in VALID_CATEGORIES:
        print(f"❌ Error: Invalid category '{category}'")
        print(f"   Valid categories: {', '.join(VALID_CATEGORIES)}")
        return None

    # Check API key
    if API_KEY == "your_api_key_here":
        print("❌ Error: Missing API key. Please set your NEWS API key.")
        return None

    # Prepare request parameters
    params = {
        "category": category.lower(),
        "country": "us",
        "apiKey": API_KEY,
        "pageSize": 5
    }

    try:
        print(f"\n📡 Fetching {category} headlines...")
        response = requests.get(BASE_URL, params=params, timeout=10)
        response.raise_for_status()

        data = response.json()
```

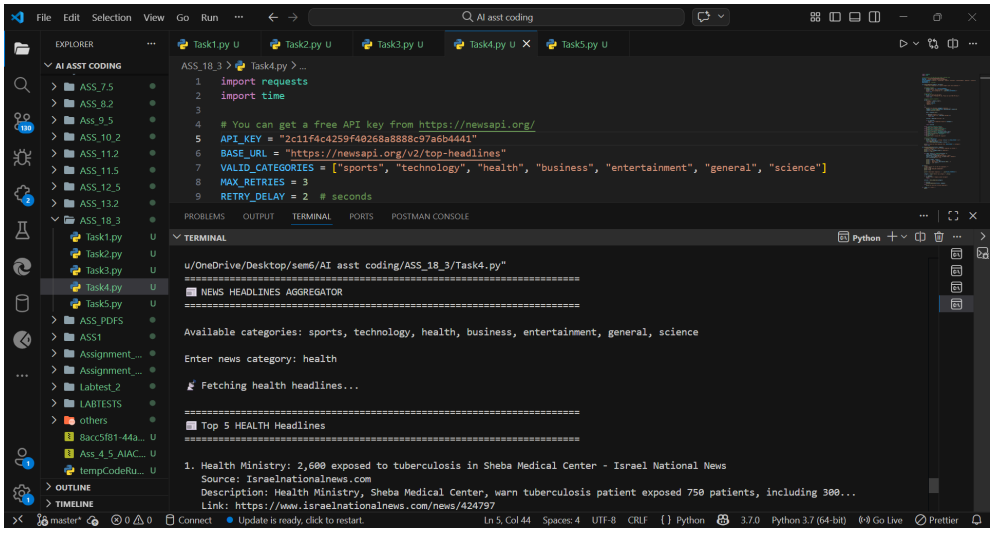
```
        if data.get("status") == "error":
            print(f"❌ API Error: {data.get('message')}")
            return None

        articles = data.get("articles", [])

        if not articles:
            print(f"⚠  No headlines found for {category}")
            return None

        return articles

    except requests.exceptions.Timeout:
        print("❌ Error: Request timeout")
    except requests.exceptions.ConnectionError:
        print("❌ Error: Connection failed")
    except requests.exceptions.HTTPError as e:
        print(f"❌ HTTP Error: {e.response.status_code}")
    except requests.exceptions.RequestException as e:
        print(f"❌ Request Error: {str(e)}")
    except ValueError:
        print("❌ Error: Invalid API response")

    # Retry mechanism
    if retries < MAX_RETRIES:
        print(f"🔄 Retrying... (Attempt {retries + 2}/{MAX_RETRIES + 1})")
        time.sleep(RETRY_DELAY)
        return fetch_headlines(category, retries + 1)

    print(f"❌ Failed to fetch headlines after {MAX_RETRIES + 1} attempts")
    return None

def display_headlines(articles, category):
    """Display headlines in a user-friendly numbered list format."""
    print(f"\n{'='*70}")
    print(f"📰 Top 5 {category.upper()} Headlines")
    print(f"{'='*70}\n")

    for index, article in enumerate(articles, 1):
        title = article.get("title", "N/A")
        description = article.get("description", "N/A")
        source = article.get("source", {}).get("name", "Unknown")
        url = article.get("url", "#")

        print(f"{index}. {title}")
        print(f"   Source: {source}")
        print(f"   Description: {description[:100]}...")
        print(f"   Link: {url}")
        print()

def main():
    """Main function to run the news aggregator."""
    print("="*70)
    print("📰 NEWS HEADLINES AGGREGATOR")
```

```
    print("="*70)

    print(f"\nAvailable categories: {', '.join(VALID_CATEGORIES)}")

    category = input("\nEnter news category: ").strip()

    if not category:
        print("❌ Error: Category cannot be empty")
        return

    articles = fetch_headlines(category)

    if articles:
        display_headlines(articles, category)
    else:
        print("\n❌ Could not retrieve headlines")

if __name__ == "__main__":
    main()
```

Explaination

💡 The news aggregator demonstrated how APIs can be used to fetch real-time information based on categories. It was observed that validating category inputs prevents unnecessary API calls. Implementing a retry mechanism improved the program's resilience against temporary failures. The output was displayed in a numbered list, making it easy for users to read. Handling missing API keys and request failures ensured smooth execution.





Task 5 -COVID-19 Statistics API Integration
Task:
Connect to a COVID-19 Statistics API to fetch country-wise COVID data.
Instructions:
• Use AI to generate Python code using the requests library
• Accept country name as user input
• Fetch key statistics such as:
o Total confirmed cases
o Total deaths
o Total recovered cases
o Active cases
• Handle API errors including:
o Invalid country name
o API rate-limit exceeded
o Network or server failures
• Implement retry or wait logic when rate limits are hit
Expected Output:
A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.
Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

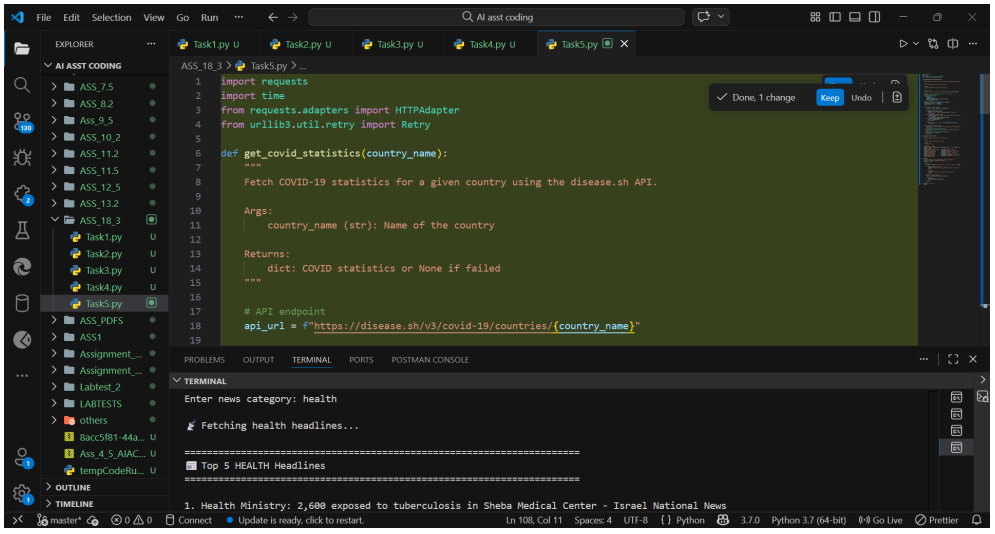
```
import requests
import time
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def get_covid_statistics(country_name):
    """
    Fetch COVID-19 statistics for a given country using the disease.sh API.

    Args:
        country_name (str): Name of the country
```

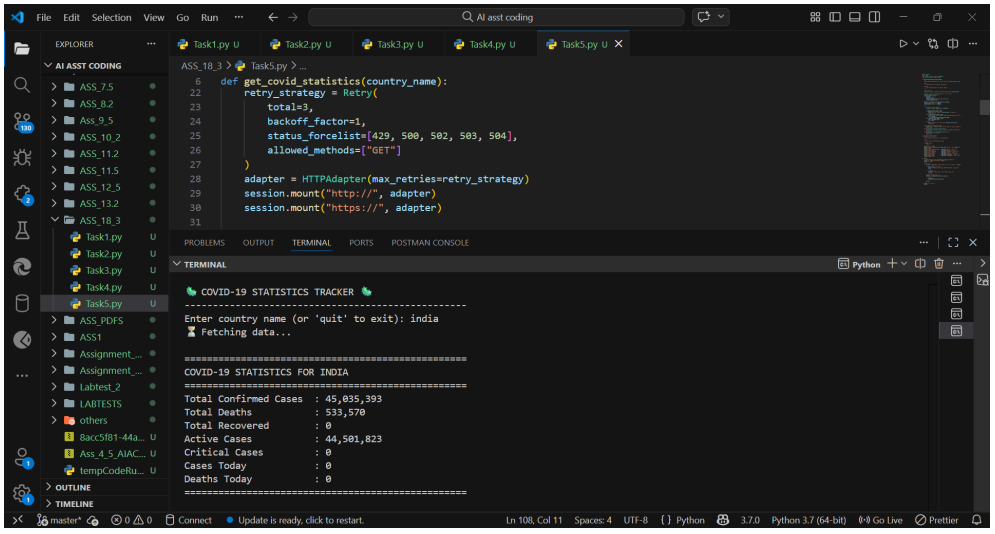
```
    Returns:
        dict: COVID statistics or None if failed
    """

    # API endpoint
    api_url = f"https://disease.sh/v3/covid-19/countries/{country_name}"

    # Setup session with retry strategy for rate limiting
    session = requests.Session()
    retry_strategy = Retry(
        total=3,
        backoff_factor=1,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET"]
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("http://", adapter)
    session.mount("https://", adapter)

    try:
        # Fetch data with timeout
        response = session.get(api_url, timeout=10)

        # Handle different status codes
        if response.status_code == 404:
            print(f"❌ Error: Country '{country_name}' not found.")
            return None
        elif response.status_code == 429:
            print("⏳ Rate limit exceeded. Waiting 60 seconds before retry...")
            time.sleep(60)
            return get_covid_statistics(country_name)
        elif response.status_code == 200:
            data = response.json()
            return data
        else:
            print(f"❌ API Error: Status code {response.status_code}")
            return None

    except requests.exceptions.Timeout:
        print("❌ Error: Request timeout. Server took too long to respond.")
        return None
    except requests.exceptions.ConnectionError:
        print("❌ Error: Network failure. Check your internet connection.")
        return None
    except requests.exceptions.RequestException as e:
        print(f"❌ Error: {str(e)}")
        return None

def display_statistics(data):
    """
    Display COVID-19 statistics in a formatted way.

    Args:
        data (dict): COVID statistics data
    """
```

```
    if data is None:
        return

    print("\n" + "="*50)
    print(f"COVID-19 STATISTICS FOR {data.get('country', 'N/A').upper()}")
    print("="*50)
    print(f"Total Confirmed Cases  : {data.get('cases', 'N/A'):,}")
    print(f"Total Deaths           : {data.get('deaths', 'N/A'):,}")
    print(f"Total Recovered        : {data.get('recovered', 'N/A'):,}")
    print(f"Active Cases           : {data.get('active', 'N/A'):,}")
    print(f"Critical Cases         : {data.get('critical', 'N/A'):,}")
    print(f"Cases Today            : {data.get('todayCases', 'N/A'):,}")
    print(f"Deaths Today           : {data.get('todayDeaths', 'N/A'):,}")
    print("="*50 + "\n")

def main():
    """Main function to run the COVID-19 statistics program."""
    print("\n🦠 COVID-19 STATISTICS TRACKER 🦠")
    print("-" * 50)

    while True:
        country = input("Enter country name (or 'quit' to exit): ").strip()

        if country.lower() == 'quit':
            print("👋 Thank you for using COVID-19 Tracker. Stay safe!")
            break

        if not country:
            print("⚠  Please enter a valid country name.\n")
            continue

        print("⏳ Fetching data...")
        statistics = get_covid_statistics(country)

        if statistics:
            display_statistics(statistics)

        print()

if __name__ == "__main__":
    main()
```

Explaination

This task showed how APIs can provide critical real-time data for global situations. It was observed that user input validation (country name) is important to avoid invalid queries. The program successfully extracted and displayed key statistics such as total cases, deaths, recovered, and active cases. Handling rate limits and network errors using retry logic ensured that the application remained functional even under API constraints.


File Edit Selection View Go Run
AI asst coding
EXPLORER
AI ASST CODING
Task1.py U
Task2.py U
Task3.py U
Task4.py U
Task5.py
ASS_18_3 > Task5.py > ...
import requests
import time
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
def get_covid_statistics(country_name):
"""
Fetch COVID-19 statistics for a given country using the disease.sh API.
Args:
country_name (str): Name of the country
Returns:
dict: COVID statistics or None if failed
"""
# API endpoint
api_url = f"https://disease.sh/v3/covid-19/countries/{country_name}"
Done, 1 change
Keep
Undo
PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE
TERMINAL
Enter news category: health
Fetching health headlines...
Top 5 HEALTH Headlines
1. Health Ministry: 2,600 exposed to tuberculosis in Sheba Medical Center - Israel National News
Update is ready, click to restart.


File Edit Selection View Go Run
AI asst coding
EXPLORER
AI ASST CODING
Task1.py U
Task2.py U
Task3.py U
Task4.py U
Task5.py U
ASS_18_3 > Task5.py > ...
def get_covid_statistics(country_name):
retry_strategy = Retry(
total=3,
backoff_factor=1,
status_forcelist=[429, 500, 502, 503, 504],
allowed_methods=["GET"]
)
adapter = HTTPAdapter(max_retries=retry_strategy)
session.mount("http://", adapter)
session.mount("https://", adapter)
PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE
TERMINAL
COVID-19 STATISTICS TRACKER
Enter country name (or 'quit' to exit): india
Fetching data...
COVID-19 STATISTICS FOR INDIA
Total Confirmed Cases : 45,035,393
Total Deaths : 533,570
Total Recovered : 0
Active Cases : 44,501,823
Critical Cases : 0
Cases Today : 0
Deaths Today : 0
Update is ready, click to restart.